Cybersecurity Internship Report

Task 2: SIEM Detection Rule Development

Development and Testing of Custom SIEM Detection Rules Using the Elastic Stack

Prepared by

Atharva Vilas Shitole

Cyber Intern

Submitted To

Elevance Skills

July 2026

Table of Contents

1. Objective

The objective of this project was to build a Security Information and Event Management (SIEM) environment using the Elastic Stack (ELK) and develop custom detection rules for identifying common cyber threats. The project focused on configuring the ELK Stack, generating simulated security events, and testing custom detection rules for **Credential Stuffing**, **DNS Tunnelling**, and **PowerShell Exploitation**. The alerts generated during testing were used to verify that the detection rules worked as expected and while gaining practical experience in log analysis, threat detection, and security monitoring.

2. Introduction

Security Information and Event Management (SIEM) plays an important role in modern cybersecurity by collecting, analyzing, and monitoring logs from different systems. It helps security teams detect suspicious activities and respond to potential threats more effectively. In this project, the Elastic Stack (ELK) was used to build a basic SIEM environment and develop custom detection rules. Simulated security events were generated to test the detection of **Credential Stuffing**, **DNS Tunnelling**, and **PowerShell Exploitation** attacks. The generated alerts were then verified in Kibana to ensure that the detection rules worked as expected.

3. Tools Used

Tool	**Purpose**
Docker	Deploy and manage ELK containers
Elasticsearch	Store and index security logs
Logstash	Process and forward log data
Kibana	Visualize logs and manage detection rules
Elastic Security	Create and monitor SIEM detection rules
Visual Studio Code	Configure Docker Compose and Logstash pipeline

4. Environment Setup

Before creating and testing the SIEM detection rules, the Elastic Stack (ELK) environment was prepared to provide a centralized platform for log collection, processing, storage, and visualization. Docker containers were used to deploy Elasticsearch, Kibana, and Logstash, while Visual Studio Code was used to configure the required files. After deployment, each service was verified to ensure the environment was ready for developing and testing custom detection rules.

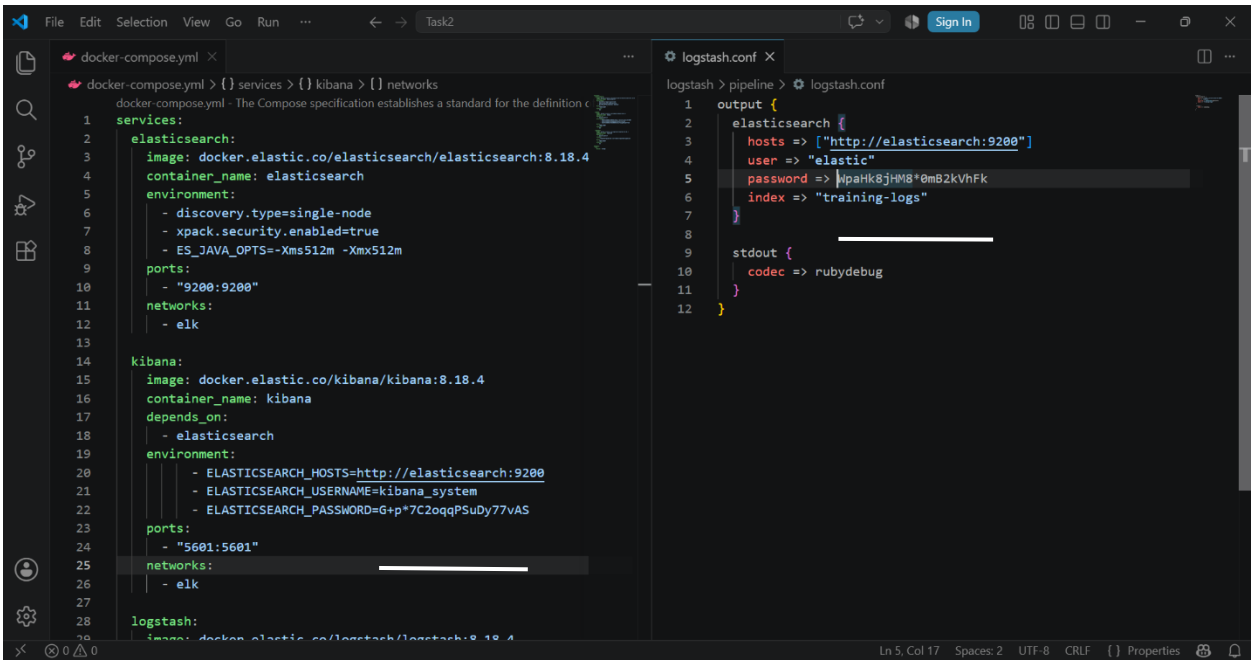

Figure 1. Docker Compose (docker-compose.yml) and Logstash (logstash.conf) configuration for deploying the ELK Stack.

The ELK Stack was configured using Docker Compose, while the Logstash pipeline was configured to process and forward simulated security events to Elasticsearch. These configuration files established the environment required for log collection and detection rule development.

```
CONTAINER ID   IMAGE                                                  COMMAND                  CREATED          STATUS          PORTS
          NAMES
afc2925b19d8   docker.elastic.co/logstash/logstash:8.18.4              "/usr/local/bin/dock…"   17 seconds ago   Up 15 seconds   0.0.0.0:5044->5044/tcp, [::]:5044->
5044/tcp   logstash
75ff2d344952   docker.elastic.co/kibana/kibana:8.18.4                 "/bin/tini -- /usr/l…"   18 seconds ago   Up 15 seconds   0.0.0.0:5601->5601/tcp, [::]:5601->
5601/tcp   kibana
b5cac4312ec7   docker.elastic.co/elasticsearch/elasticsearch:8.18.4   "/bin/tini -- /usr/l…"   18 seconds ago   Up 16 seconds   0.0.0.0:9200->9200/tcp, [::]:9200->
9200/tcp   elasticsearch
```

Figure 2. Running ELK Stack containers in Docker Desktop.

After completing the configuration, the ELK Stack services were started using Docker. The successful execution of the Elasticsearch, Kibana, and Logstash containers confirmed that the environment was deployed correctly and ready for further configuration and testing.

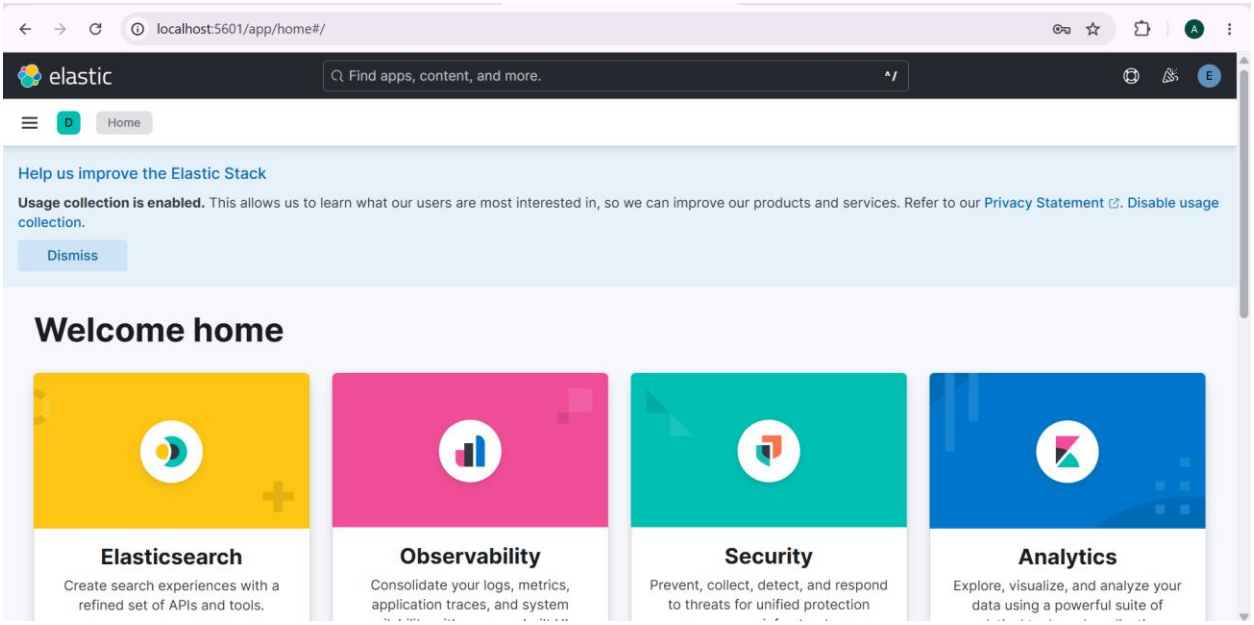

Figure 3. Kibana dashboard after successful deployment of the ELK Stack.

After starting the ELK services, Kibana was accessed through the web browser to verify that the deployment was successful. The dashboard loaded correctly, confirming that Kibana was connected to Elasticsearch and the environment was ready for log analysis and SIEM rule development.

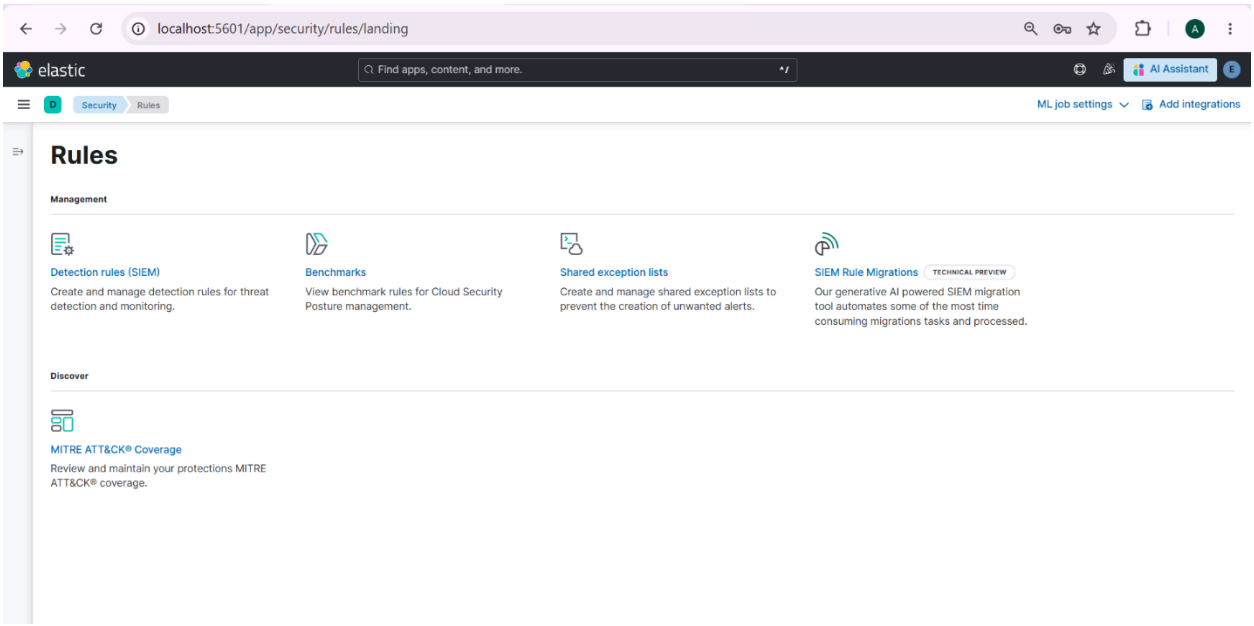

Figure 4. Elastic Security Rules module used to access and manage SIEM detection rules.

After completing the ELK Stack setup, the Rules module in Elastic Security was accessed through Kibana. This interface provides access to SIEM detection rules, exception lists, and related security features, and was used as the starting point for creating the custom detection rules developed in this project.

5. Credential Stuffing Detection

Credential Stuffing is a cyberattack in which attackers attempt to gain unauthorized access to user accounts by using stolen username and password combinations. To simulate this scenario, sample failed login events were indexed into Elasticsearch, and a custom SIEM detection rule was created in Kibana. The rule was then tested to verify that matching events generated security alerts successfully.

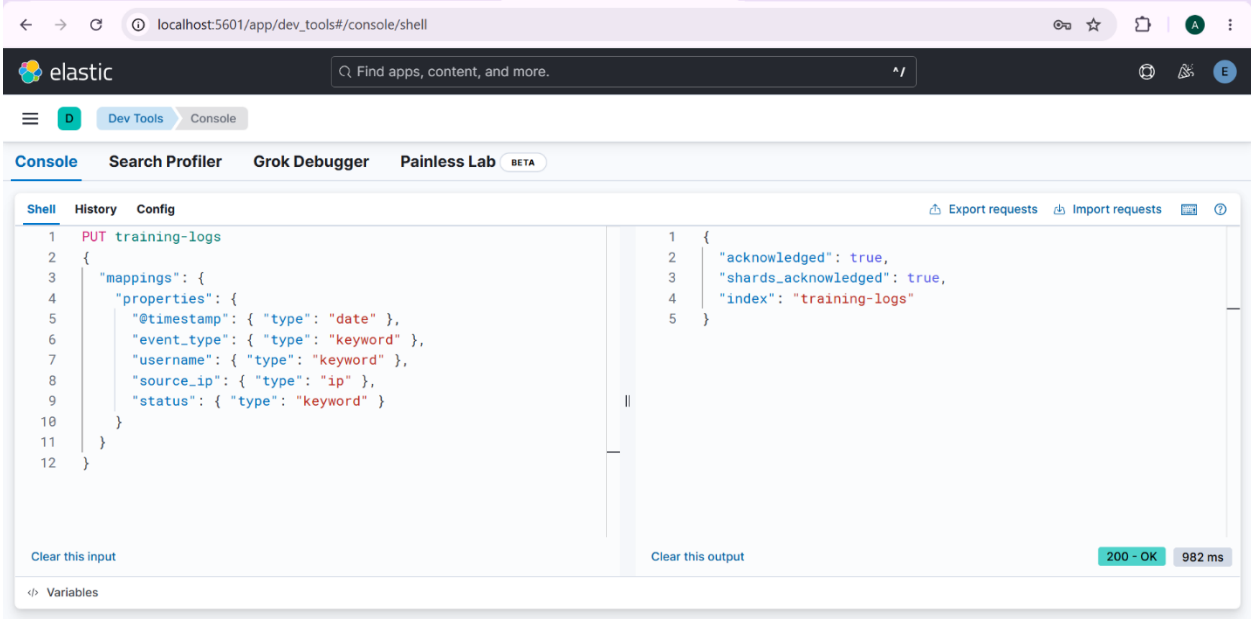

Figure 5. Creation of the **training-logs** index using Kibana Dev Tools.

The training-logs index was created in Elasticsearch to store the simulated authentication events used during testing. This index served as the primary data source for the custom Credential Stuffing detection rule, enabling Elasticsearch to store and retrieve the events required for rule validation.

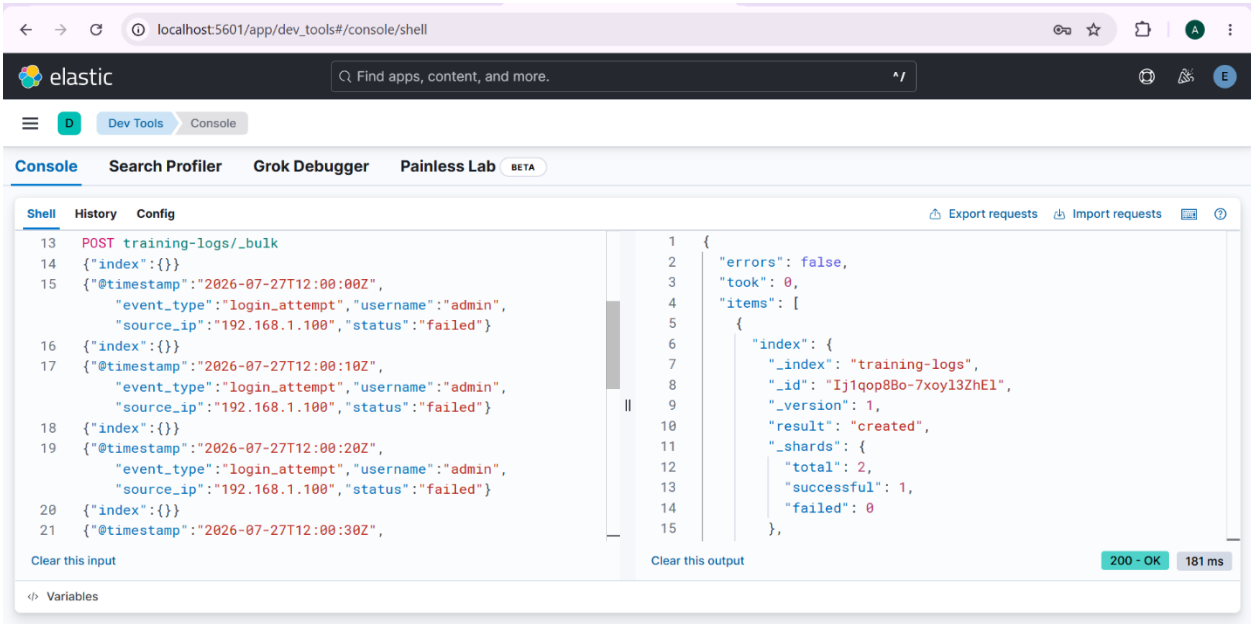

Figure 6. Inserting simulated failed login events into the training-logs index using the Elasticsearch Bulk API.

Simulated failed login events were inserted into the training-logs index using the Elasticsearch Bulk API. These events represented multiple unsuccessful authentication attempts and were used to simulate a Credential Stuffing attack for testing the custom SIEM detection rule.

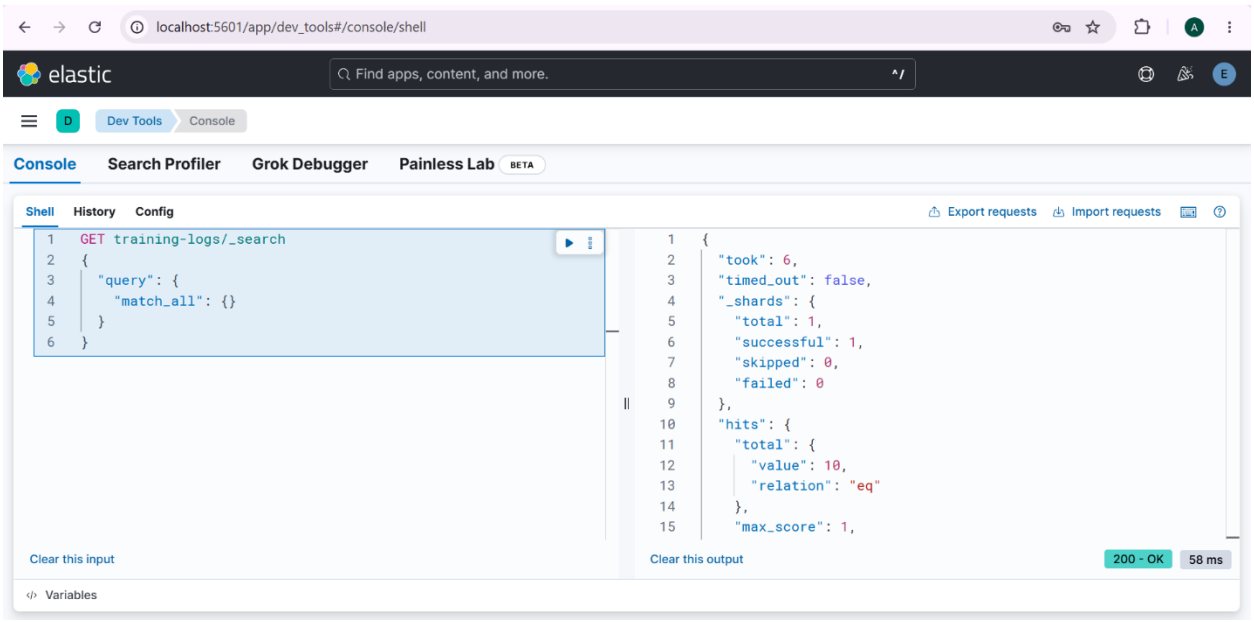

Figure 7. Verification of the simulated login events stored in the training-logs index.

Once the simulated login events were added, the training-logs index was searched to verify that the data had been indexed successfully. The returned results confirmed that the events were available in Elasticsearch and could be used for testing the custom Credential Stuffing detection rule.

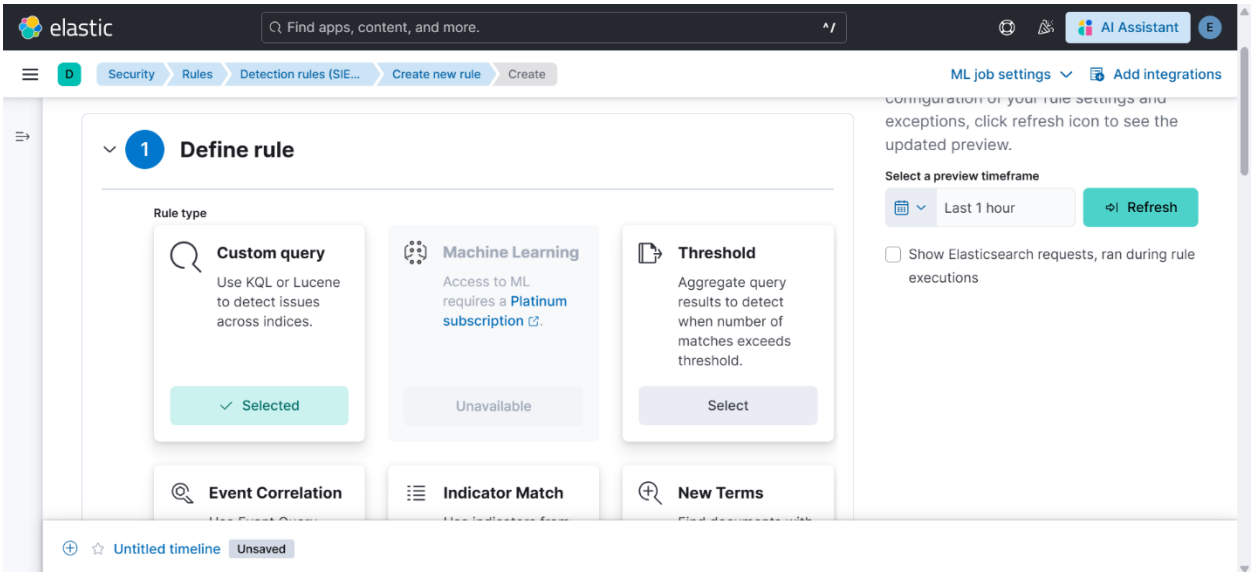

Figure 8(a). Selecting the Custom Query rule type for creating a new SIEM detection rule.

The rule creation process was initiated in Kibana by selecting the **Custom Query** rule type. This option allows custom detection logic to be created using Elasticsearch queries, making it suitable for detecting simulated Credential Stuffing activities.

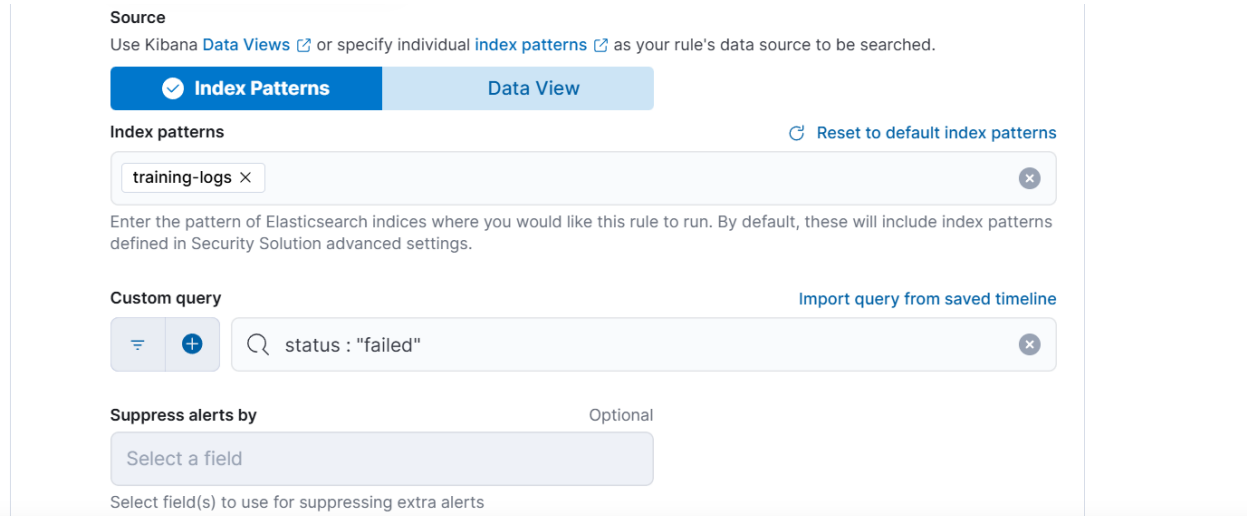

Figure 8(b). Configuring the custom Credential Stuffing detection rule using the **training-logs** index.

The detection rule was configured to monitor failed login attempts stored in the training-logs index. A custom query, **status: "failed",** was used to identify unsuccessful authentication events and generate alerts whenever the defined conditions were met.

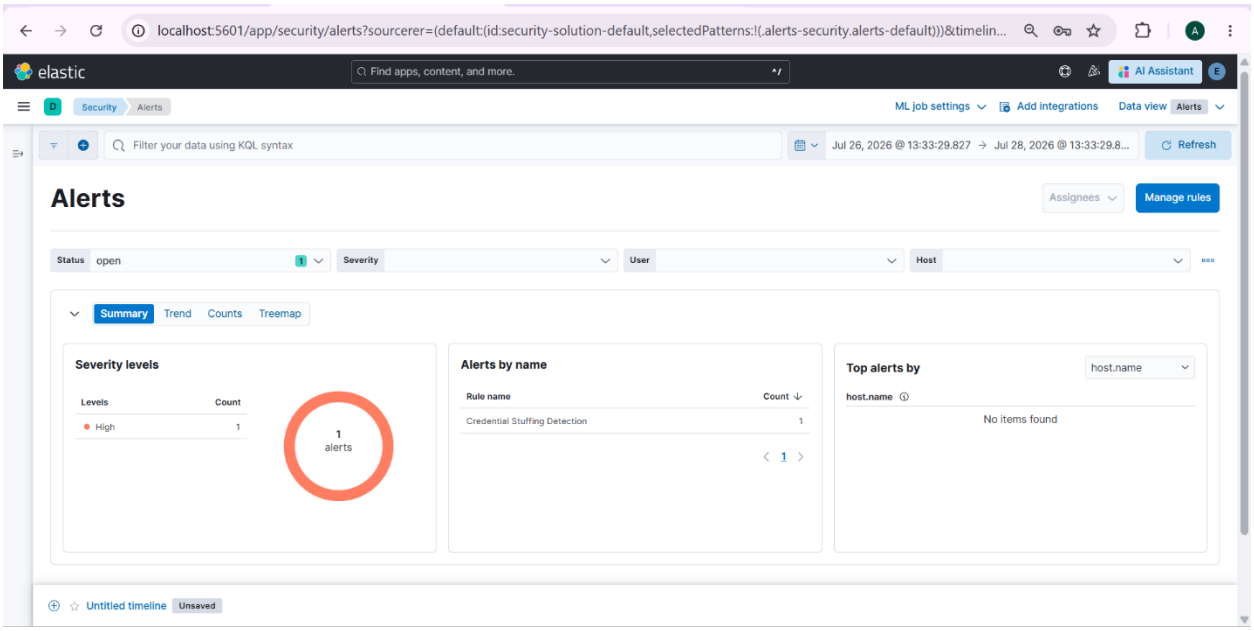

Figure 9. Successful alert generated by the custom Credential Stuffing detection rule.

After the custom detection rule was executed, Kibana generated an alert based on the simulated failed login events. The alert appeared in the Security dashboard with a high severity level, confirming that the rule successfully detected the simulated Credential Stuffing activity. This validated that the configured SIEM rule was functioning as expected.

5.1 Chapter Summary

The Credential Stuffing detection rule was successfully implemented and validated using the Elastic Stack. Simulated failed login events were indexed into Elasticsearch, monitored through a custom SIEM detection rule, and successfully generated alerts in Kibana. This implementation demonstrated the complete workflow of developing, testing, and validating a custom SIEM detection rule for detecting suspicious authentication activities.

6. DNS Tunnelling Detection

DNS Tunnelling is a cyberattack technique in which attackers exploit the Domain Name System (DNS) protocol to establish a covert communication channel between an infected system and an external server. Since DNS traffic is commonly allowed through network security devices, attackers may use it to bypass security controls, transfer sensitive information, or communicate with command-and-control (C2) servers. To simulate this scenario, sample DNS events were indexed into Elasticsearch, and a custom SIEM detection rule was created in Kibana. The rule was then tested to verify that suspicious DNS queries generated security alerts successfully.

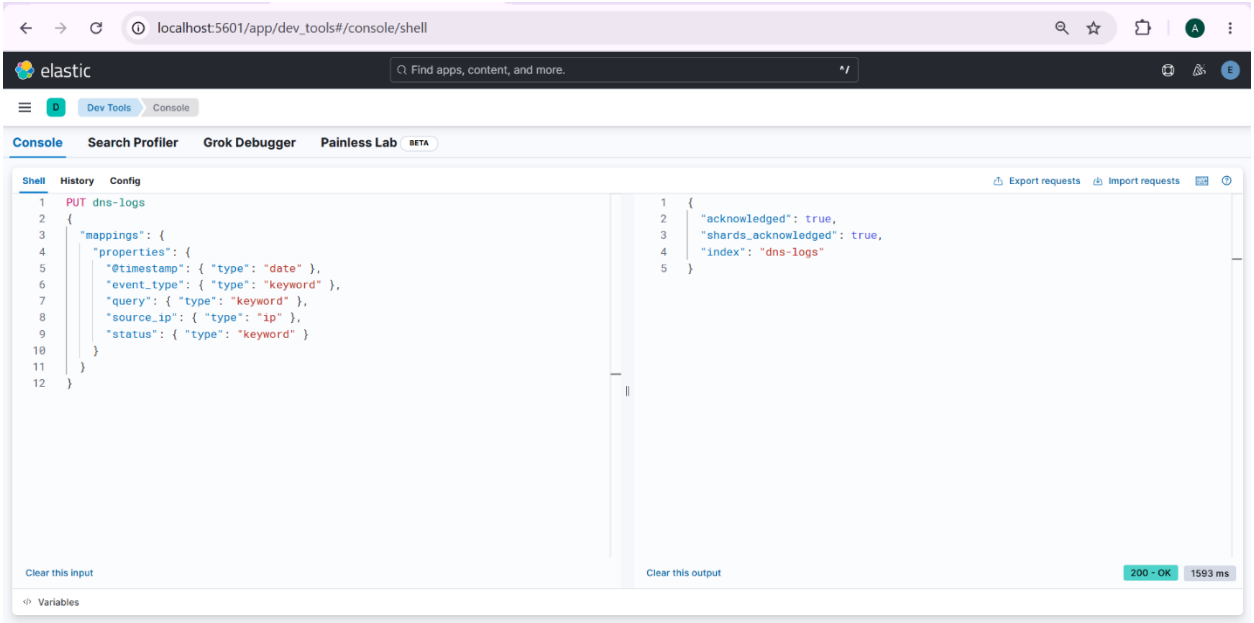

Figure 10. Creation of the **dns-logs** index using Kibana Dev Tools.

The **dns-logs** index was created in Elasticsearch to store the sample DNS events used during testing. The index mapping defined important fields such as **timestamp, DNS query name, event category, and source IP address**, providing a structured data source for the DNS Tunnelling detection rule.

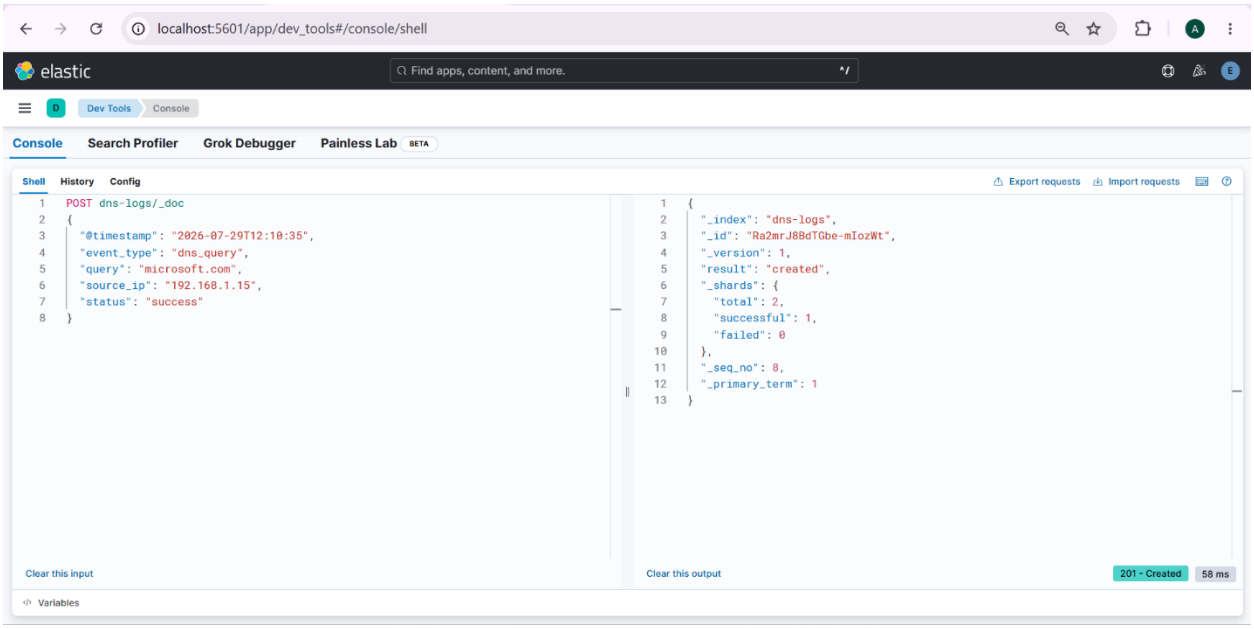

Figure 11. Insertion of simulated DNS events into the dns-logs index using Kibana Dev Tools.

The simulated DNS events were inserted into the **dns-logs** index to represent both normal and suspicious DNS activity. These sample records provided the required dataset for testing the DNS Tunnelling detection rule and verifying its ability to identify malicious DNS queries.

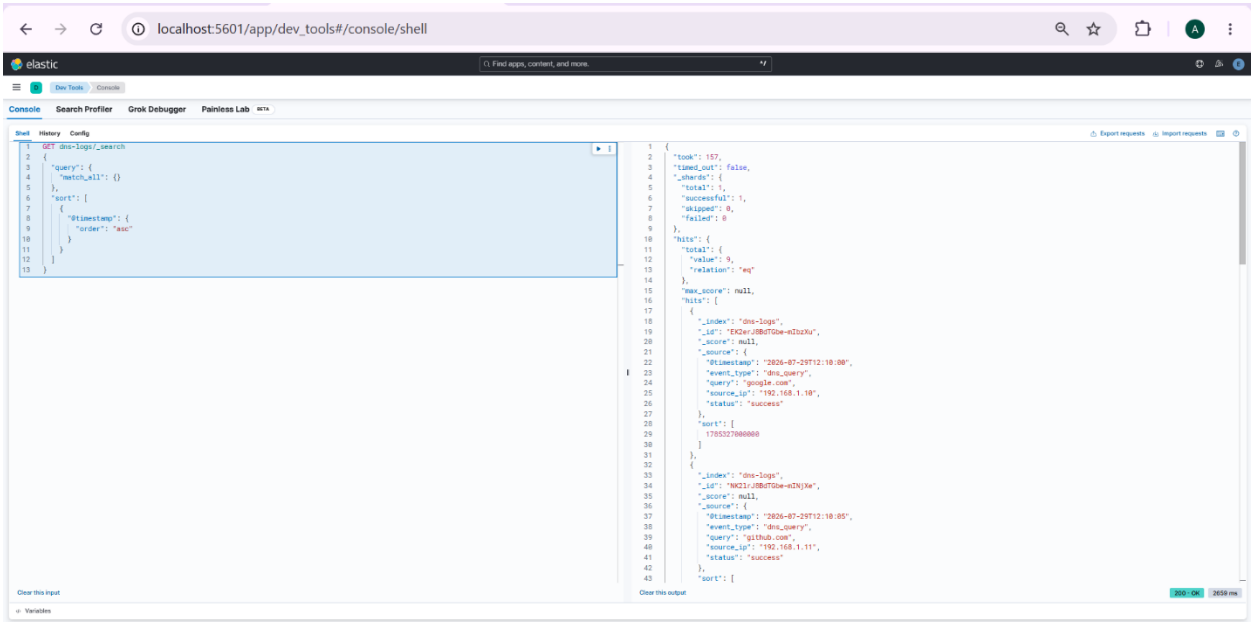

Figure 12. Verification of DNS events stored in the dns-logs index.

The indexed DNS events were verified using a search query in Kibana Dev Tools. The search results confirmed that the DNS records were successfully stored in Elasticsearch and were available for analysis by the DNS Tunnelling detection rule.

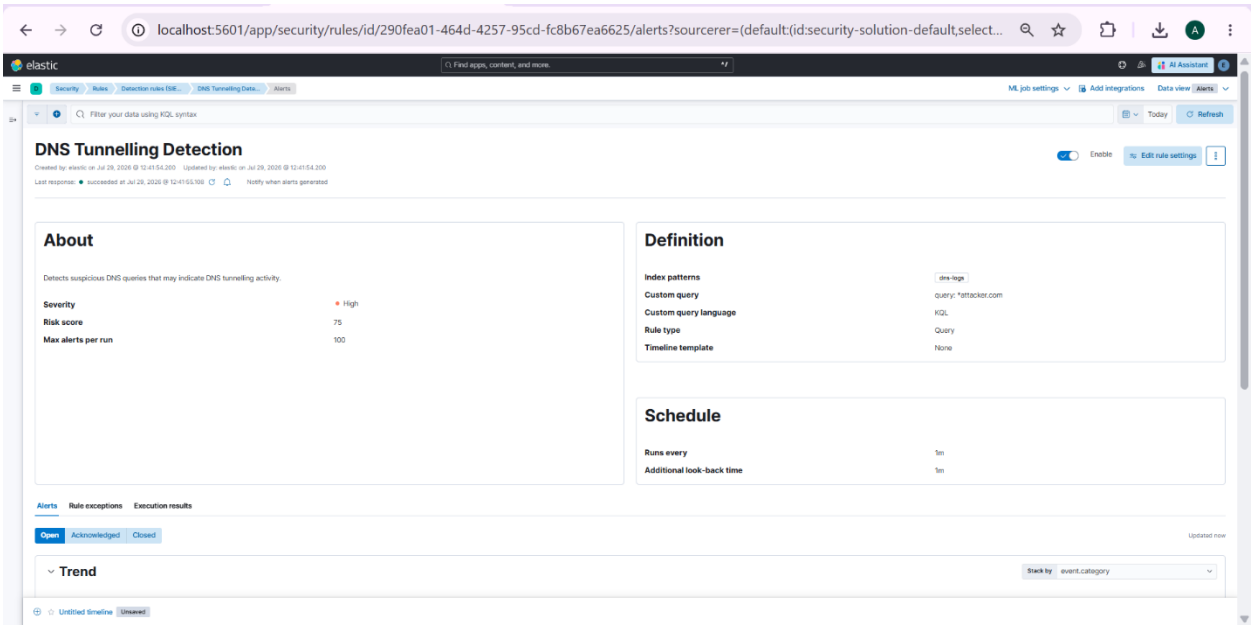

Figure 13. Configuration of the **DNS Tunnelling Detection** rule in Elastic Security.

A custom detection rule named **DNS Tunnelling Detection** was created in Elastic Security to monitor the **dns-logs** index for suspicious DNS queries. The rule was configured with a custom query and scheduled to run periodically, to detect potential DNS tunnelling activity.

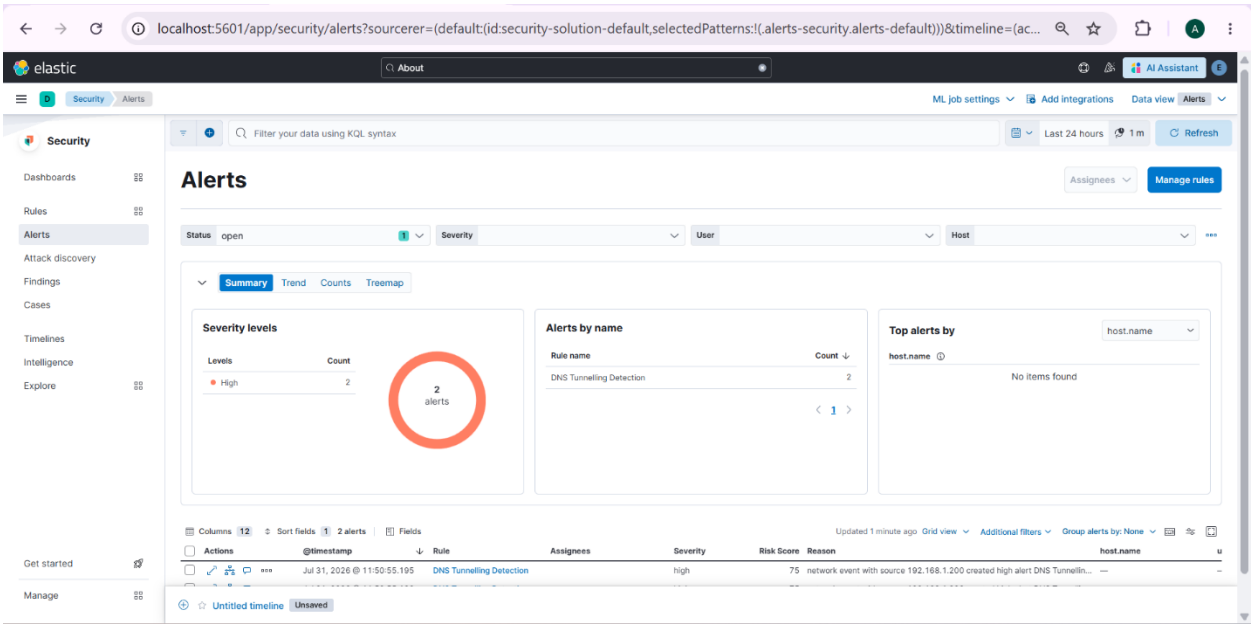

Figure 14. Generation of DNS Tunnelling detection alerts in Elastic Security.

The configured detection rule successfully identified the simulated suspicious DNS queries and generated security alerts in the Elastic Security dashboard. The generated alerts confirmed that the rule was functioning correctly and was capable of detecting potential DNS tunnelling activity based on the defined conditions.

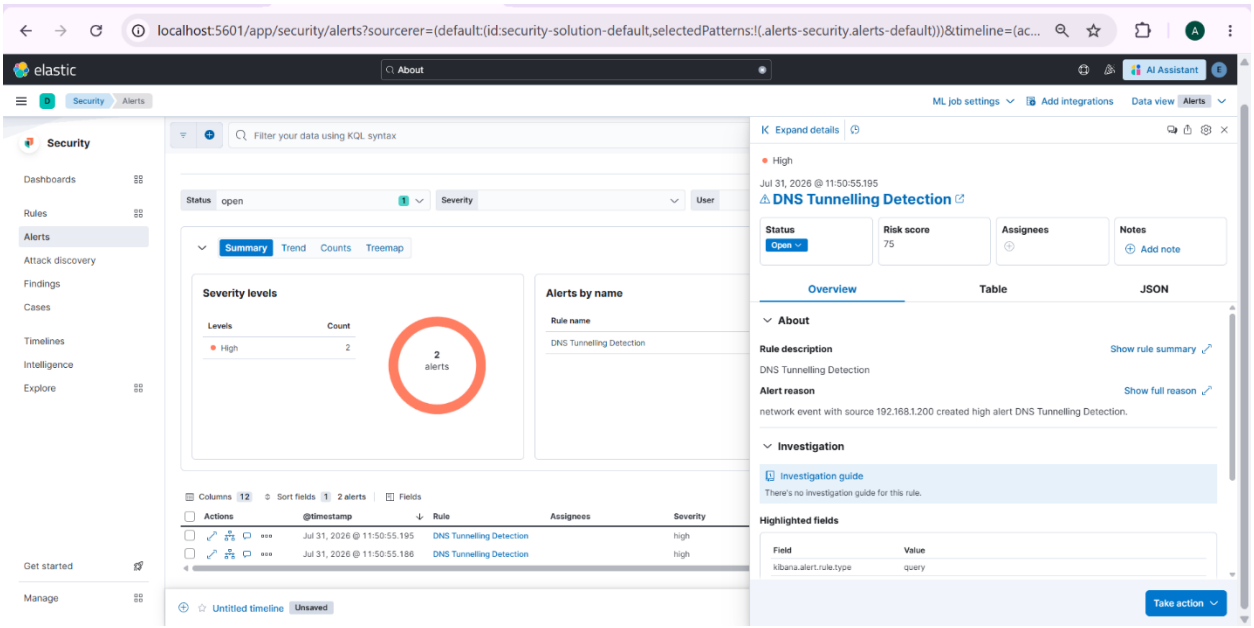

Figure 15. Investigation details of the generated DNS Tunnelling detection alert.

The generated alert was opened to review its detailed information, including the alert reason, severity level, risk score, source IP address, and associated detection rule. These details provide security analysts with the necessary information to investigate and validate suspicious DNS activity effectively.

6.1 Chapter Summary

The DNS Tunnelling detection task demonstrated how Elastic Security can identify suspicious DNS activity using custom detection rules. Sample DNS events were indexed into Elasticsearch, verified successfully, and monitored using a custom detection rule configured in Elastic Security. The rule detected suspicious DNS queries and generated high-severity alerts, demonstrating the effectiveness of SIEM-based monitoring in identifying potential DNS tunnelling attacks.

7. PowerShell Exploitation Detection

PowerShell is a powerful command-line scripting tool built into Microsoft Windows that is widely used for system administration and automation. However, attackers frequently misuse PowerShell to execute malicious commands, download harmful payloads, bypass security mechanisms, and perform unauthorized activities on compromised systems. To simulate this scenario, sample PowerShell process events were indexed into Elasticsearch, and a custom SIEM detection rule was created in Kibana. The rule was then tested to verify that suspicious PowerShell commands generated security alerts successfully.

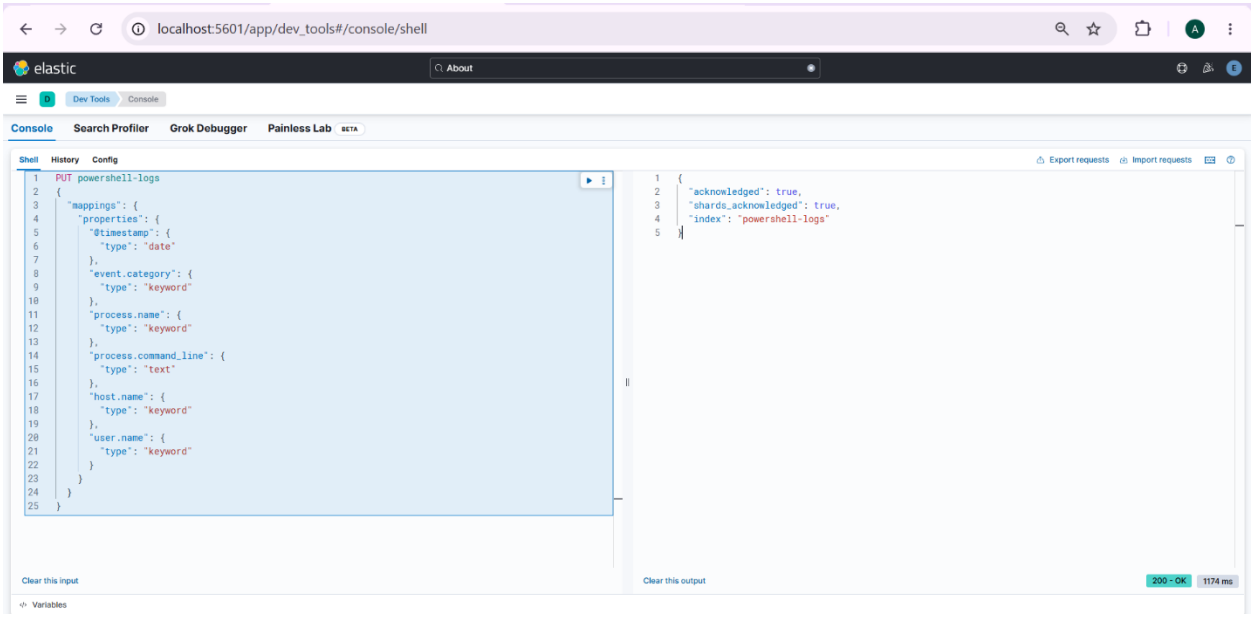

Figure 16. Creation of the **powershell-logs** index using Kibana Dev Tools.

The **powershell-logs** index was created in Elasticsearch to store the sample PowerShell process events used during testing. The index mapping defined important fields such as **timestamp, event category, process name, process command line, host name, and user name**, providing a structured data source for the PowerShell Exploitation detection rule.

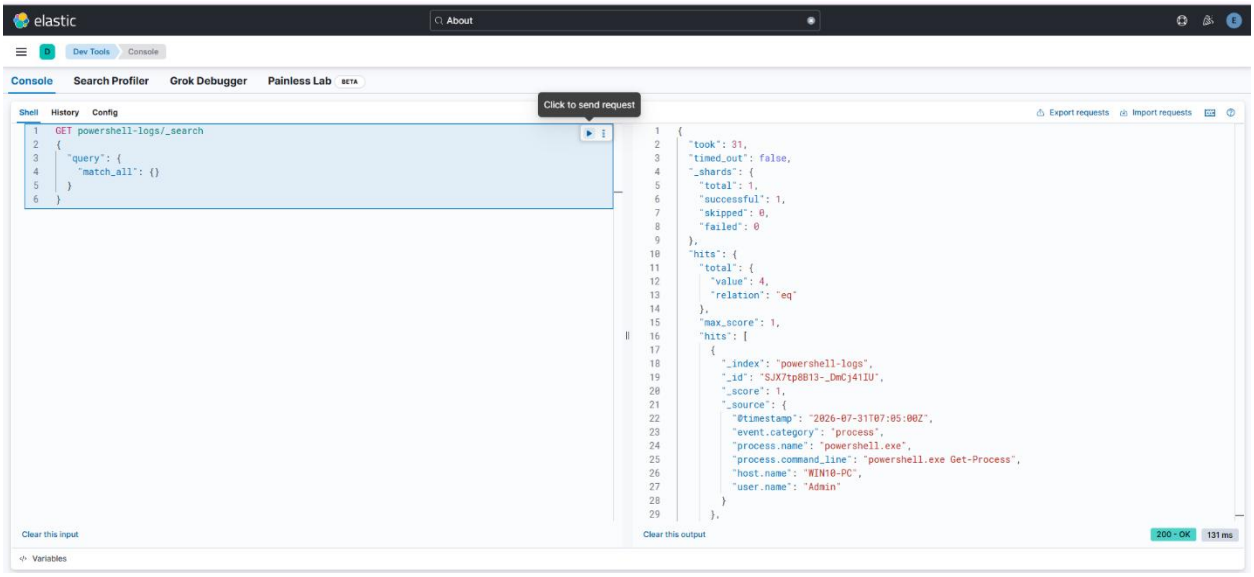

Figure 17. Insertion of simulated PowerShell events into the **powershell-logs** index.

The simulated PowerShell process events were inserted into the **powershell-logs** index to represent both normal and suspicious PowerShell activities. These sample records provided the required dataset for testing the PowerShell Exploitation detection rule and verifying its ability to identify potentially malicious PowerShell commands.

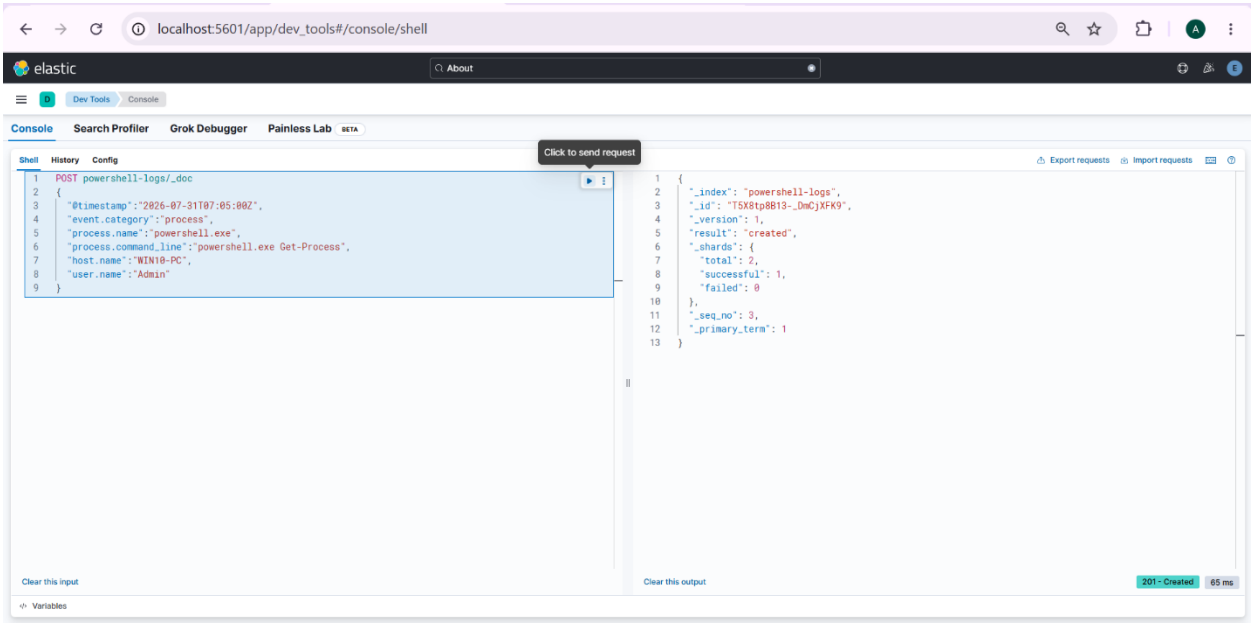

Figure 18. Verification of PowerShell events in the **powershell-logs** index.

The indexed PowerShell events were verified using a search query in Kibana Dev Tools. The search results confirmed that the PowerShell process records were successfully stored in Elasticsearch and were available for analysis by the PowerShell Exploitation detection rule.

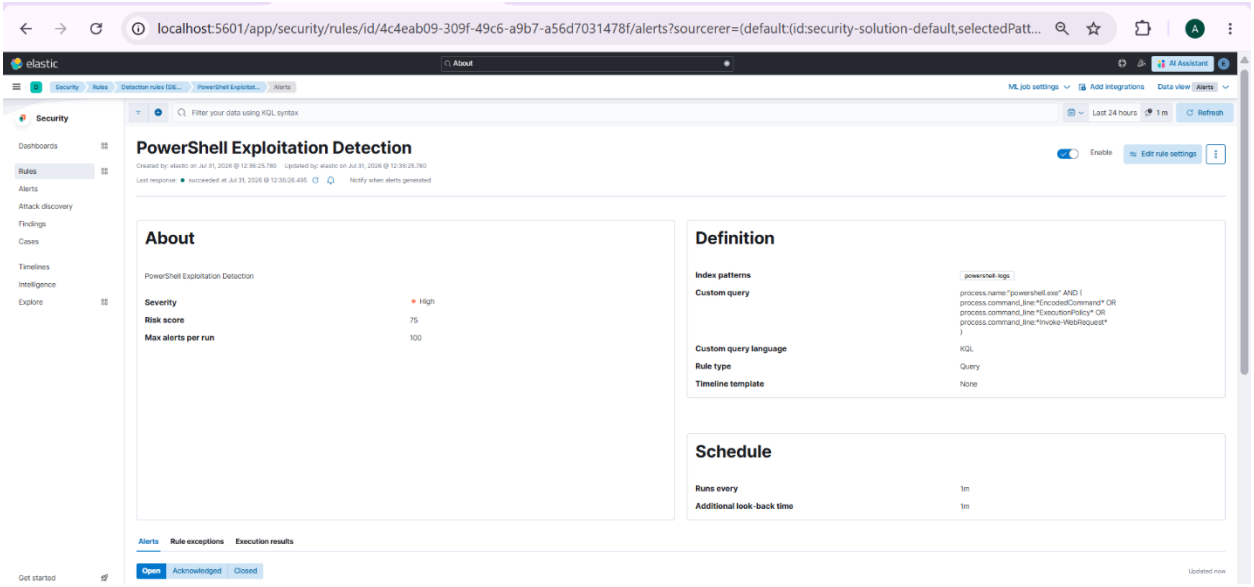

Figure 19. Configuration of the **PowerShell Exploitation Detection** rule in Elastic Security.

A custom detection rule named **PowerShell Exploitation Detection** was created in Elastic Security to monitor the **powershell-logs** index for suspicious PowerShell activities. The rule was configured with a custom query to detect potentially malicious PowerShell commands and scheduled to run periodically to detect possible PowerShell exploitation attempts.

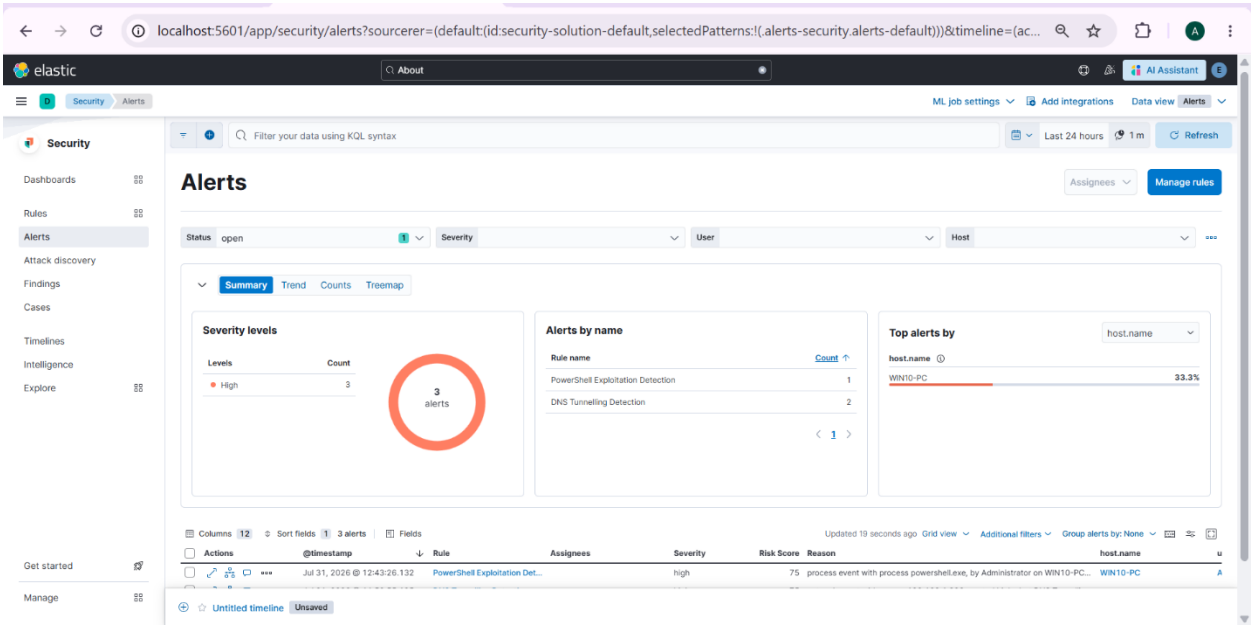

Figure 20. Generation of PowerShell Exploitation Detection alerts in Elastic Security.

The configured detection rule successfully identified the simulated suspicious PowerShell activities and generated security alerts in the Elastic Security dashboard. The generated alert confirmed that the rule was functioning correctly.

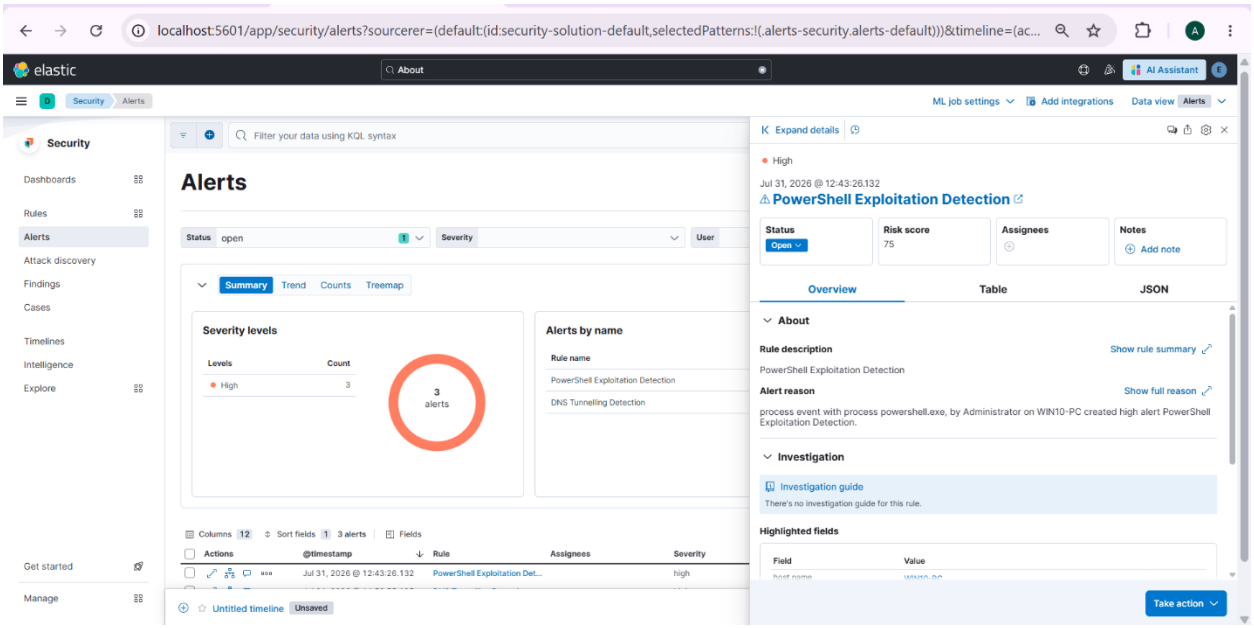

Figure 21. Investigation details of the generated **PowerShell Exploitation Detection** alert.

The generated alert was opened to review its detailed information, including the alert reason, severity level, risk score, host name, user name, and the associated detection rule. These details provide security analysts with the necessary information to investigate suspicious PowerShell activities and validate potential PowerShell exploitation attempts effectively.

7.1 Chapter Summary

In this chapter, PowerShell Exploitation detection was implemented using Elastic Security by creating a dedicated PowerShell log index, ingesting sample PowerShell process events, and configuring a custom detection rule. The rule successfully identified suspicious PowerShell commands and generated high-severity alerts, demonstrating the effectiveness of SIEM-based monitoring in detecting potential PowerShell exploitation attempts.

8. Testing and Validation

The implemented SIEM correlation rules were tested using simulated attack scenarios to validate their functionality. The generated alerts confirmed that the configured detection rules successfully detected the simulated attack techniques and provided relevant information for security analysis and incident investigation.

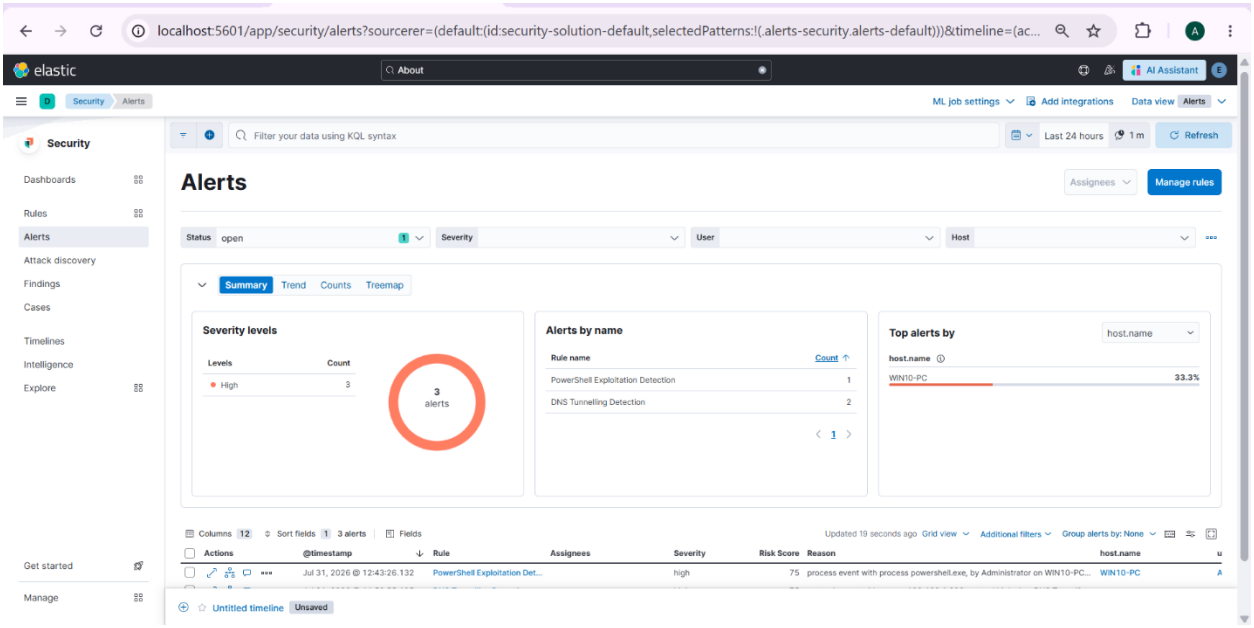

Figure 22. Detection alerts generated for the implemented SIEM correlation rules.

The Elastic Security dashboard displayed alerts generated by the implemented detection rules after processing the simulated attack events. The successful generation of alerts confirmed the effectiveness of the custom SIEM correlation rules in detecting Credential Stuffing, DNS Tunnelling, and PowerShell Exploitation attacks.

9. Conclusion

This project involved implementing and testing custom SIEM correlation rules using the Elastic (ELK) Stack to detect Credential Stuffing, DNS Tunnelling, and PowerShell Exploitation attacks. Simulated attack events were successfully detected, and the generated alerts confirmed that the configured detection rules worked as expected. Overall, this project provided practical experience in SIEM, log analysis, detection engineering, and security monitoring while improving the understanding of real-world cyberthreat detection.